ROR: Using Reads for Fast and Efficient Data Repair in Erasure-Coded Storage Systems

Marc Tranzer
marc.tranzer@inria.fr
Inria, Univ. Rennes, CNRS, IRISA
Rennes, France

Shadi Ibrahim
shadi.ibrahim@inria.fr
Inria, Univ. Rennes, CNRS, IRISA
Rennes, France

Abstract

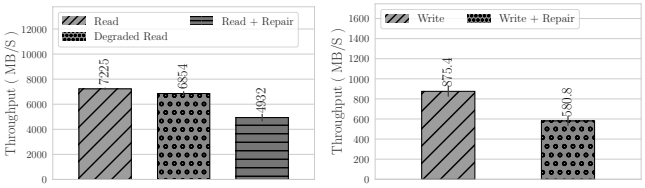
Many storage systems use erasure coding (EC) to ensure high data availability. However, EC presents the key issue of high data repair costs. Due to the large amount of data exchanged during the repair process, the repair can significantly impact read and write operations. Previous studies have demonstrated the feasibility of lazy repair, which involves delaying repairs to consider a few failures instead of just one, thereby reducing repair bandwidth. In this paper, we demonstrate the potential of exploiting reads and employing lazy repair through the design and implementation of Repair-On-Read (ROR). ROR caches recovered chunks during degraded reads and uses them later to enable fast and efficient repairs. We also implemented RORP, which performs encoding during regular reads to recover and cache missing parity chunks. Our evaluation on CephFS shows that ROR and RORP with RS(10, 4) can reduce repair bandwidth by up to 63% and 88%, respectively, resulting in reduced contention and improved read throughput by up to 90% in the case of failure.

CCS Concepts: • **Computer systems organization → Redundancy**; • **Information systems → Distributed storage**.

Keywords: Distributed Storage, Erasure Coding, Data Repair, Fault Tolerance, Ceph

1 Introduction

The adoption of erasure coding (EC) in storage systems has shifted from niche archival applications to a generalized storage paradigm for warm and even hot data. Distributed storage systems typically implement EC at the client and cluster levels. The former is mainly used in content sharing systems (e.g., P2P storage systems [1, 2, 3]) and data analytic clusters (e.g., HDFS [4, 5]) where clients reside inside the cluster. However, most modern storage systems, including Ceph, Azure Storage, and Facebook's F4 [6, 7, 8] implement EC at the cluster level, i.e., they perform encoding and decoding inside the cluster. For each block, one node (primary node) is designated to perform encoding and decoding and to distribute or collect the chunks when writing or reading, respectively.



(a) Read Throughput **(b)** Write Throughput

Figure 1. Aggregated throughput of 10 concurrent reads and writes of 10 files of 10 GB (Testbed Section 4.1)

EC can provide data availability and durability similar to that of replication but with significantly less storage overhead. However, it incurs higher costs in terms of data exchange, I/O, and computation during the repair process (i.e., high repair bandwidth). Unlike replication, where a block is simply sent from one of the remaining replicas to a new node, under EC, chunks of a block must be collected and decoded. Then, the missing chunk is sent to a new node. According to Facebook's traces, repair traffic can reach 10-20% of the total cluster traffic [9]. This makes read and write operations extremely sensitive to the traffic introduced by repair and the resulting resource contention. As shown in Figures 1a and 1b read and write performance degrades by 32% and 34%, respectively, when data repair runs in parallel with those operations.

Previous studies have focused on making data exchange network-aware to accelerate data transfer during repairs [10, 11, 12, 13]; on new erasure code families that reduce the size of data repair, though this comes at the cost of more complex encoding and decoding [14, 15] or increased storage overhead [16, 7, 9]; or on how to maintain high read performance during repairs [11, 17, 18]. Another approach to reduce the size of data repair is lazy repair [19, 20, 21]. Instead of triggering the repair process for a single node failure, this approach considers the fact that node failures are mostly uncorrelated (in Facebook, node failure is uncorrelated more than 90% of

This work is licensed under a Creative Commons Attribution 4.0 International License.
CHEOPS '26, April 27–30, 2026, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2604-0/26/04
https://doi.org/10.1145/3805687.3806259

the time [9]), and accordingly, it delays the repair process so that multiple failures can be repaired simultaneously.

Given the feasibility of lazy repair and the high frequency of read operations in production clusters (even for warm data, such as photo-sharing services, data access frequency peaks at twice a week [22]), in this paper, we argue that read operations can be used to reconstruct missing chunks, thereby reducing the amount of data exchanged (repair bandwidth) when the repair process is triggered. On the one hand, under failures, many reads may require that a block be reconstructed if one of the original chunks is missing (i.e., degraded reads). Therefore, by delaying repair and saving these data, we can avoid reconstructing them again during the repair process. On the other hand, the performance of degraded reads is comparable to that of regular reads [10, 23]. As shown in Figure 1a, despite having one less machine to read data from, degraded reads are less than 5% slower than regular reads under concurrent reads. Therefore, delaying repair will not significantly impact the overall throughput of reads in the cluster.

We propose Repair-On-Read (ROR), a new approach that uses degraded reads to reduce the amount of data required for repairs. ROR caches and indexes recovered chunks in primary nodes during degraded reads. Since the same primary node is responsible for reading and repairing a particular block, the cached chunks are later used for repair, thus avoiding the need to reconstruct the whole block again. In addition, we evaluate a more aggressive design: RORP, which extends ROR to parity chunks. In addition to repairing and caching missing original chunks during degraded reads, RORP identifies any missing parity chunk and re-encodes the blocks during regular reads. We implement ROR and RORP in CephFS and evaluate their effectiveness in reducing data exchange (repair bandwidth) and improving energy efficiency, as well as their efficiency in reducing the impact of the repair process on read and write operations. Our evaluation results on the Grid'5000 testbed [24] show that ROR and RORP with the Reed-Solomon code, RS(10, 4), reduce repair bandwidth by up to 63% and 88%, respectively, compared to the baseline, making the repair 44% and 82% more energy efficient. In addition, the results show that ROR and RORP can increase read throughput by up to 90% and decrease energy consumption by 33% in a cluster with frequent reads after a failure. This is achieved by reducing the contention induced by the repair. Finally, our results demonstrate the applicability, effectiveness, and efficiency of ROR to other EC families, such as Locally Repairable Code (LRC) – a widely adopted network code for reducing the repair bandwidth [25, 16, 7].

In summary, this paper makes the following contributions:

- We present the design and implementation of ROR and RORP, two lazy repair approaches that leverage reads to reduce repair bandwidth.
- We quantitatively evaluate and demonstrate the effectiveness and efficiency of ROR and RORP under various scenarios, workloads, recovery delays, and network settings.
- We show that although ROR and RORP are designed for RS codes, they can be applied to and extended for other EC families.

The rest of the paper is organized as follows: Section 2 provides an overview of CephFS, including data write, read, and repair under erasure coding. Section 3 describes the design and implementation of ROR. Section 4 discusses the evaluation results. Section 5 reviews related work, and Section 6 concludes the paper.

2 Erasure Codes in CephFS

CephFS is an open-source file system that has been widely adopted by major vendors, including OVHcloud [26] and IBM [27]. It supports replication and EC as fault tolerance techniques to ensure the availability of the data. Currently, CephFS supports four EC families including Reed-Solomon codes, recovery-efficient codes such as Locally Repairable Codes (LRC), and regenerating codes (Clay).

CephFS employs the CRUSH algorithm [28] to place and locate the EC chunks across each OSDs. At scale, Crush is CPU-intensive; therefore, CephFS creates pre-computed placement groups (PGs) that contain the list of OSDs to store the EC chunks. Each PG assigns a primary OSD responsible for performing the erasure coding operations at the cluster level. By default, files are divided into 4 MB blocks, which are stored in object storage daemons (OSDs). Each block is assigned to a PG associated with a primary OSD that divides it into N chunks and then encodes those chunks to generate K parity chunks. These chunks (original and parity) are stored across N + K different OSDs. A block can be reconstructed by retrieving any N chunks, with an extra decoding step if a parity chunk is selected.

CephFS detects a failed OSD if no heartbeat is received for 15 minutes. After an additional 10 minutes, it marks the node as offline and initiates the repair process [29]. Figure 2 describes the repair process. First, CephFS updates each degraded PG (PG with missing chunks) to assign a new OSD and "optionally" a new primary OSD. Then, the primary OSD, which is usually responsible for the repair and degraded reads, repairs the block in the background. Specifically, it retrieves N surviving chunks from N OSDs, decodes them, reconstructs the lost chunks, and forwards the recovered chunks to their new locations. Note that to recover a node, multiple PGs with different primaries are repaired in parallel.

3 Repair-On-Read

We design and implement Repair-On-Read (ROR), a new approach that leverages recovered chunks during degraded reads to reduce repair bandwidth without impacting read

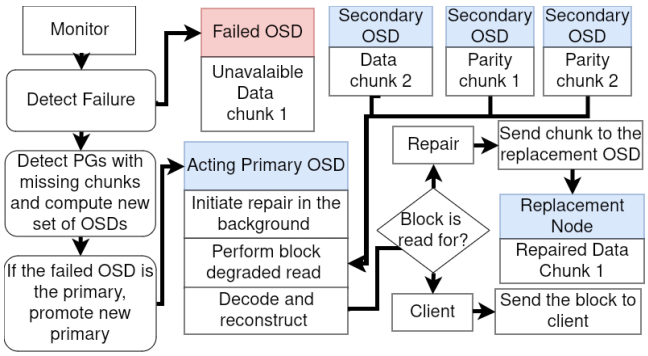


Figure 2. Repairing a RS(3,2) PG in CephFS

performance or data durability. We design ROR to meet the following goals:

- **High fault tolerance:** ROR is based on the lazy repair approach, which has proven to provide high fault tolerance in practice [19, 20, 21]. Repairs can be delayed and triggered only after multiple failures occur, especially since failures are usually uncorrelated in production storage systems. Moreover, ROR can build on previous work targeting the monitoring of fault tolerance in erasure-coded storage systems [19, 30].
- **Low repair bandwidth:** For cold data, as demonstrated in [19] applying lazy repair increases the chance of multiple failures on the same EC stripe and reduces repair bandwidth. This is because repairing a block with two missing chunks requires collecting as many chunks as repairing a single failure. For warm and hot data, reads are frequent. ROR uses degraded reads and cached recovered chunks. This significantly reduces the repair bandwidth.
- **Fast repair:** The use of recovered chunks during degraded reads and the reduction of repair bandwidth enable a fast repair process.
- **Low impact on system performance:** Delaying repairs results in a high number of degraded reads. Degraded reads perform slightly slower than regular reads [10, 23, 31], thus lowering the impact of ROR on system throughput. On the other hand, delaying repairs preserves network, CPU, and I/O resources for read and write operations, thereby ensuring high performance. Finally, as the amount of data to be repaired decreases significantly, ROR can result in low resource contention and reduce the impact of repairs on read and write operations.
- **Acceptable memory overhead:** ROR only caches the chunks needed by repair and distributes these chunks across the whole cluster.
- **Generality and Applicability**: ROR is implemented on top of CephFS; however, it can be adapted to any erasure-coded storage system that implements EC at the cluster level. In addition, ROR can be used with other EC families to further reduce the repair bandwidth and its impact.

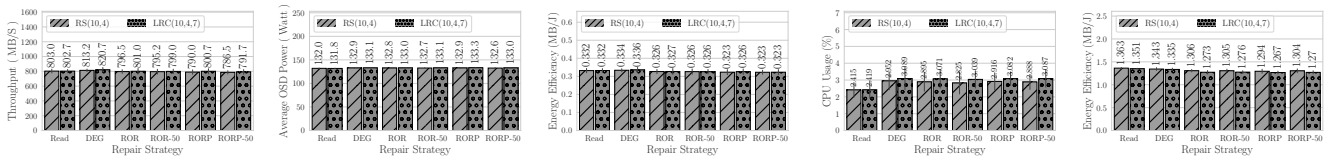
Implementation. ROR is implemented in Ceph ("reef" version) and performs the two following functions: (i) Stores any recovered chunks in the primary node cache during the degraded read. (ii) Uses locally cached chunks when the repair is triggered instead of performing a full block repair.
(1) Recovering the chunks on degraded reads. We configure a local cache disk within each OSD node. In our experiment, we create a 5 GB RAM disk, but our design can work with cold storage devices as well. This disk stores the repaired chunks encountered during degraded reads and indexes them as $< block_name > _ < chunk_index >$.

Next, we modify CephFS source code to utilize this cache. When a block is read, CephFS identifies which chunks are available and which are necessary to reconstruct the block. If any original chunks are missing, CephFS performs degraded reads. It retrieves any N surviving chunks (original or parity) and then decodes the missing chunks to reconstruct the block. ROR lists the decoded chunks, caches and indexes any original chunks that were identified as unavailable by CephFS. We also introduce a variant of ROR that repairs parity chunks (named **RORP**): During both reads and degraded reads, RORP checks if the PG group contains any missing chunks (original or parity). If a parity chunk is missing, RORP re-encodes the block and caches the missing chunk. This enables the complete repair of both original and parity chunks during reads but incurs additional CPU costs.
(2) Repairing and restoring the chunk to a new OSD. When the repair is triggered, ROR checks if the missing chunks are available and indexed in the primary node cache locally. Otherwise, it performs a default CephFS repair. In either case, the repair happens on the same acting primary OSD that performed the degraded read. Therefore, under ROR and RORP, any recovered chunk during a degraded read (a read) will be available in the local cache of the OSD responsible for the repair. This is not the case with default repair, in which the primary retrieves N surviving chunks (typically, one will be local and N-1 will be retrieved from other OSDs) and then decodes them. Once the missing chunks are identified or recovered, they are forwarded to the replacement OSD and written to the disk.
Recovery cost. Under RS(10,4), the recovery step with the default repair requires collecting 9 chunks externally (0 under ROR if the chunk is cached), while the forwarding step only requires sending 1 chunk under both.

4 Evaluation

4.1 Methodology

Testbed. We demonstrate the potential of ROR on the Grid' 5000 testbed [24]. We use the "Paradoxe" cluster with 18 OSD machines and up to 20 clients. We also use 2 machines in the "Parasilo" cluster to store the monitor and the metadata server daemon. Each machine in the Paradoxe cluster is equipped with 2 CPUs of 26 cores (Intel Xeon Gold 5320) and SSDs



(a) Throughput **(b)** Power **(c)** Energy Efficiency **(d)** CPU Usage (5 clients) **(e)** Energy Efficiency (5 clients)

Figure 3. Performance evaluation of 10 GB file reads with 1 and 5 concurrent clients on the Paradoxe cluster (10 Gbps network bandwidth).

(SATA HP MK001920GZXRC). The nodes are equipped with wattmeters that measure their power consumption every 20 ms. We set the network bandwidth to 1 Gbps and 10 Gbps between OSDs using "`TC`".

Platform. Our experiment uses Debian 11 OS machines with an updated version of Ceph version 18 "reef", which includes the core ROR (RORP) functionality. We control and monitor the repair workflow and apply lazy repair using Python scripts.

Baseline. Our baseline is the production code RS(10,4) [9] as well as the common locality code LRC(10,4,7) [25] with the default CephFS repair strategy. We compare both codes with and without ROR and RORP enabled. We also tested RS(10,5), which offers increased durability, as well as three common codes: RS(8,2), RS(6,3), and RS(9,3) [7, 32, 33].

Evaluation Metrics. Our main metrics are the throughput (aggregated throughout for concurrent operations) and energy efficiency. Energy efficiency is computed as the ratio of the amount of data read (repaired) to the total energy consumed, which is the sum of the energy consumed by all OSDs and network traffic. We also evaluate the repair time and repair bandwidth. We calculate the energy consumed by network traffic based on the model presented in [34].

4.2 Evaluating read and repair operations.

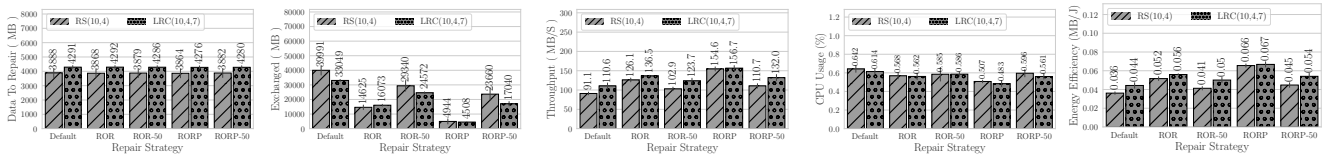
Experiment Workflow. For each evaluated approach, we run 5 iterations. During each experiment, we first store 5 raw binary files, each 10 GB. The files are generated by the "`dd`" command and then divided into 4 MB blocks by CephFS. Second, we perform a standard read on a healthy cluster. Then, we emulate a node failure by stopping an OSD and marking it as offline. Then, we perform a degraded read with the default, ROR, and RORP approaches on each file. We also evaluate ROR-50 and RORP-50, in which only 50% of the nodes have ROR and RORP enabled. The latter allows us to assess the gain when not all data can be repaired within a specific timeframe. Finally, we trigger the repair process by adding a fresh OSD and measuring the repair performance under each approach.

It is important to note that reads and degraded reads are executed independently of the repair process. This decoupling allows us to evaluate the overhead and benefits of ROR and RORP in isolation. Specifically, this allows us to measure their impact on degraded read performance and the advantages of cached data during subsequent repairs.

Single Read. Figures 3a, 3b, and 3c show the read throughput, power consumption, and energy efficiency of 1 client reading a 10GB file under RS(10,4) and LRC(10,4,7) under read, degraded read (DEG), ROR, and RORP.

First, we observe that the read and degraded read throughput are similar when reading one single file, which highlights that the decoding time under RS is minimal and is offset by a better read balance, as demonstrated in [35]. Second, we observe that ROR and RORP incur minor performance overheads of 2% and 3%, respectively, during degraded and regular read operations. For ROR, this is due to the caching of the repaired chunks, which occurs before the delivery of the block. This can be mitigated, however, by performing caching and block delivery in parallel. Note that the performance overhead is slightly higher under RORP due to the time required for re-encoding missing parity chunks. In terms of energy consumption, degraded reads with or without ROR and RORP consume an additional 1 watt of power compared to regular reads due to the energy used for decoding the missing chunks. As a result, ROR and RORP require 3% more energy than regular reads.

Concurrent Reads. Figures 3d and 3e show the CPU usage and energy efficiency of 5 clients reading 5 files of 10 GB with the same tested codes and approaches. We observe similar throughput for reads, degraded reads without ROR, and degraded reads with ROR with respective rates of 3966, 3979, and 3896 MB/s. However, degraded reads with and without ROR consume 25% more CPU than regular reads. This difference is more pronounced in concurrent reads because multiple decoding and encoding operations are performed in parallel. This results in an additional 3 watts of power consumption and 4% lower energy efficiency under ROR compared to reading on a healthy cluster. Note that with 10 clients (Figure 1a), the cluster experiences more contention. This contention, in addition to the loss of 5.5% of the nodes, leads to a performance drop of up to 5%, which explains why degraded reads perform worse than regular reads. This performance drop is due to reduced aggregated cluster bandwidth. However, this degradation is much smaller than that



(a) Data Repair Volume **(b)** Data Exchanged **(c)** Throughput **(d)** CPU Usage **(e)** Energy Efficiency

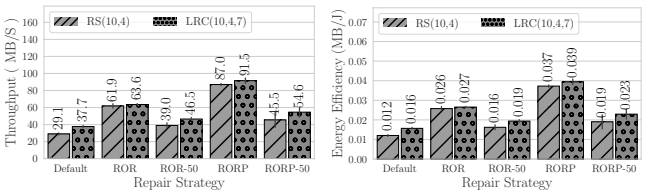
Figure 4. Performance evaluation of repairing five 10 GB files in the Paradoxe cluster with a 10 Gbps network bandwidth.

caused by contention when repair is performed immediately. Overall, the degraded reads induced by lazy repair and the extra steps in ROR have a limited impact on read performance and energy compared to the baseline.

Single Repair. Figures 4a, 4b, and 4c show the data repair volume, data exchanged, and repair throughput when repairing five 10 GB files. First, RS(10,4) with the ROR approach requires 10% less data to be repaired than LRC(10,4,7) due to reduced storage overhead. Second, our evaluation shows significant gains from ROR and RORP. They reduce the amount of data exchanged by 63% and 88%, respectively, compared to the baseline. Third, ROR-50 reduces the amount of data exchanged by 27% (42% under RORP-50) compared to the RS(10,4) baseline and 11% (29% under RORP-50) compared to the LRC(10,4,7) baseline. This highlights that ROR is beneficial even when data are not fully read before repair. Fourth, our results reveal that ROR can also be beneficial with locality codes: ROR-50 with LRC exchanges 16% less data than ROR-50 with RS(10,4). Finally, the reduction in data transfer translates to higher repair throughput. Specifically, ROR and RORP repair 38% and 69% faster than the default RS(10,4), as well as 14% and 40% faster than the default LRC(10,4,7).

In terms of energy consumption, ROR and especially RORP use much less CPU during repair compared to the baseline, which translates to 3 and 6 fewer watts used, respectively. This is due to (i) reduced network consumption and (ii) the fact that ROR does not have to decode because the original missing chunks are already stored locally. RORP also avoids re-encoding the missing parity chunks. Interestingly, this means that the extra CPU cost of degraded reads can be offset during repair. Overall, RORP with RS(10,4) is 83% more energy efficient than the baseline and 50% more energy efficient than LRC(10,4,7) during repair.

Impact of network bandwidth. Bandwidth is often limited, especially across racks and in geo-distributed storage systems [9, 36, 37] and optimizing repair for such environments is an active area of research [13, 12]. In a bandwidth-limited network, the cost of data transfer becomes predominant compared to the EC computation cost and the cost of reading and writing to disk. Figures 5a and 5b show the repair throughput and energy efficiency of repairing five 10 GB files with different repair approaches and 1 GB network bandwidth.



(a) Throughput **(b)** Energy Efficiency

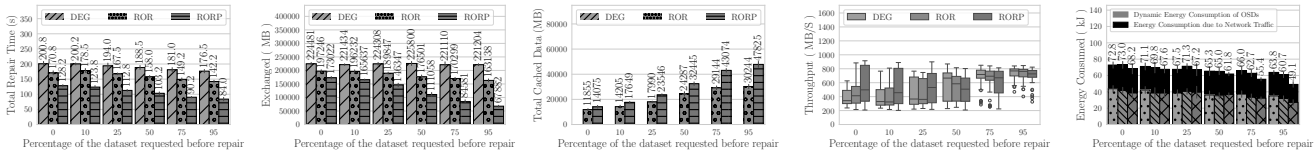
Figure 5. Performance evaluation of repairing 5 files of 10 GB in the Paradoxe cluster with a 1 Gbps network bandwidth.

ROR with RS(10,4) achieves 2.13× (3× under RORP) the throughput of baseline and 1.63× (2.35× under RORP) the throughput of LRC(10,4,7). These gains are much higher than those observed in a 10 Gbps network. In terms of energy, ROR and RORP consume less power than the baseline, and the energy efficiency increases significantly: from 1.43× (1.82× under RORP) in 10 Gbps to 2.15× (3.2× under RORP) in 1 Gbps.

Generalization to other erasure code parameters. Table 1 presents the energy efficiency of reading three 10 GB files concurrently, as well as the energy efficiency and amount of data exchanged during a subsequent repair of the three 10 GB files using different repair approaches under RS(10,4), RS(10,5), RS(6,3), RS(9,3), and RS(8,2).

We observe that our repair approach can be applied to any EC parameters. First, we can see that the ROR and RORP present a slight but consistent decrease in the energy efficiency of degraded reads. For instance, ROR introduces a 3 – 5% energy efficiency penalty compared to the baseline across various EC parameters.

Second, the amount of data exchanged by ROR depends on the ratio of original to parity chunks and is therefore better with RS(8,2) and RS(9,3). Meanwhile, RORP shows a similar data exchange across all codes. Compared to the baseline, the reduction depends on how the code is designed for fast repair. RS(6,3) requires 38% less data to be exchanged during repair compared to RS(10,4). Therefore, ROR and RORP reduce the data exchanged by 51% and 84%, respectively, compared to the RS(6,3) baseline, while the reduction is more pronounced with RS(10,4), reaching 61% and 90%.



(a) Repair Time **(b)** Repair Data Exchanged **(c)** Cumulative Data Cached **(d)** Read Throughput Per Files **(e)** Dynamic Energy

Figure 6. Performance evaluation of reading and repairing 60 files of 4 GB in the Paradoxe cluster with a 10 Gbps network. Each file request is separated by 3 seconds, and the repair starts after 0, 10, 25, 50, 75, 95% of the files have been requested. We use RS(10,4)

RS parameters (n,k)	**10,4**	**10,5**	**6,3**	**9,3**	**8,2**
Read EE (MB/J)	0.751	0.754	0.770	0.747	0.769
Degraded Read (DR) EE	0.764	0.763	0.773	0.759	0.771
DR with ROR EE	0.732	0.745	0.756	0.740	0.751
DR with RORP EE	0.73	0.740	0.747	0.737	0.752
Repair bandwith (MB)	23025	24209	14166	16302	14311
ROR Repair bandwith	8983	10508	6973	5981	4665
RORP Repair bandwith	2309	2465	2129	2955	2251
Repair EE (MB/J)	0.029	0.029	0.035	0.030	0.031
ROR Repair EE	0.043	0.037	0.049	0.04	0.046
RORP Repair EE	0.052	0.049	0.047	0.05	0.047

Table 1. Performance evaluation of concurrent reads and a subsequent repair of three 10 GB files under 10 Gbps network bandwidth.

4.3 Mixing frequent reads with repair

Experiment Workflow. We run an experimental workflow in order to mimic the behavior of the baseline, lazy repairs, and ROR in a frequently accessed CephFS cluster – configured with RS(10,4) –under failure. First, we store a dataset of 60 files of 4 GB in our cluster. Then, we stop one OSD and mark it as offline. We do not replace it; instead, we let the surviving OSDs in the cluster collectively store the repaired chunk. Finally, we initiate 60 read operations across 20 concurrent clients, with a 3-second interval between successive requests. Hence, the last file is requested 180 seconds after the first one. We measure the read and repair throughput with and without ROR (RORP) after 0%, 10%, 25%, 50%, 75%, and 95% of the files have been requested (therefore, after 0, 18, 45, 90, 135, and 171 seconds).

Results. Figures 6a and 6d depict the repair time and the read throughput when running our experiment. We observe that the read time and the repair time progressively decrease as more delay is applied to the repair. The repair time – under default – decreases from 200.8 seconds for immediate repair to 176.5 seconds when the maximum repair delay is applied. Meanwhile, the read throughput increases from an average of 378 MB/s to 703 MB/s. This is expected because each operation mostly runs independently of the others when the maximum delay is applied. It is important to note that delaying the repair process under the default increases the execution time from 275 to 348 seconds and that the cluster experiences a period of low utilization during which the repair runs without contention.

Figures 6b and 6c show the amount of data exchanged during the repair and the cumulative amount of data that is cached under ROR and RORP. First, we observe that the gain from ROR is limited, which can be explained by the cluster size. As RS(10,4) requires using 14 nodes out of 17, the PG combinations are limited, and many degraded PGs require re-balancing in order to build a suitable new PG that does not already exist. This is evident from the amount of data written: CephFS writes 55 GB of data even though an average of only 18 GB of data is lost when we remove one OSD. We deduce that a degraded PG may have 2 or 3 failing/re-balancing chunks and therefore have a high probability that this includes at least one parity. The previous observation does not affect RORP results because it also repairs parity chunks. It is important to note that this situation is specific to CephFS and to the size of our cluster.

Second, ROR and RORP gains are visible even without delaying repairs, with, respectively, a 13% and 22% reduction in data exchange, a 15% and 36% reduction of the repair time, as well as a 12% and 40% increase in the read throughput compared to the baseline. While repairs are running, many blocks can be read and missing chunks can be cached before being repaired, reaching 25% of the total missing chunks with RORP. In general, faster and more efficient repairs result in less contention when reading. This is particularly true for the last retrieved files. For example, an immediate repair under RORP is completed in 128 seconds, while the last file read begins after 180 seconds. This makes reading the last files twice as fast as the baseline. On longer delay, the read throughput under RORP matches that of naive lazy repair but with the additional benefit of having the repair finish in 84 seconds. This results in a total execution time of 262 seconds, which is less than the default of 275 seconds and much less than the naive lazy repair of 348 seconds.

In order to provide a clearer comparison of the different approaches in terms of energy consumption, we evaluate the

dynamic energy consumption of the cluster. This metric is computed by summing the energy of each OSD – after subtracting the idle power baseline – and including the energy due to network traffic. As shown in Figure 6e, the energy consumption of RORP is reduced by 6% without delay and by up to 32% when delaying the repair. This is due to the faster execution time and the reduced energy consumption of the network traffic during the repair process.

5 Related Work

One line of research focuses on introducing new code constructions to reduce the repair bandwidth of EC code. The most notable codes are: (i) Locally Repairable Codes (LRCs): LRC codes are widely used in production systems such as Microsoft Azure [7] and Facebook's HDFS cluster [9]. LRC introduces local parity to RS codes, enabling the repairs of a single failure with half the amount of chunks. However, LRC uses more storage than RS. (ii) Regenerating codes (RGs): RGs include the Product-Matrix code [14], Clay code [15] and NCBlob [38], among others. RG reduces repair bandwidth compared to RS codes. But it exhibits very high encoding and decoding costs. *ROR can be applied to these codes, thus further reducing repair bandwidth and its impact on system throughput.*

A second line of research focuses on reducing the impact of repairs on storage systems. Pertin *et al.* [17] demonstrate that encoding and decoding reduce the available CPU for other operations, thereby decreasing their performance. They introduce the Mojette code that uses less CPU than RS with ISA-L. Zhou *et al.* [10] schedule parallel repairs on nodes with the least traffic at a given time, even if these nodes are not part of the EC stripe. In addition, several studies [11, 12, 13] explore repair pipelining techniques to reduce cross-rack traffic, where bandwidth is typically constrained. These approaches perform successive partial repairs within a rack. They also implement different scheduling and routing techniques to reduce contention when exchanging repaired chunks across racks. *While these solutions improve repair performance and system performance, they still cause high repair bandwidth during repair. ROR complements these solutions by reducing repair bandwidth to further improve performance and system throughput.*

Another approach is to perform lazy repairs by consolidating multiple repairs at the same time. This reduces the overall network usage of successive repairs and, consequently, the impact of repairs on the storage system [20, 21]. *Our work extends lazy repairs to warm and hot data by investigating the benefits of reads to reduce the amount of exchanged data.*

6 Conclusion and Future Work

We presented Repair-On-Read (ROR), a new repair strategy that uses lazy repair and leverages degraded reads to cache and repair any chunks belonging to failed nodes. Our evaluation using CephFS demonstrates that ROR is effective and efficient, reducing repair bandwidth by up to 88% and reducing contention on reads.

Future work will focus on addressing the following issues: (i) Explore the cost of using HDDs and SSDs to store repaired chunks, and investigate new ways to reduce the overhead and impact of data caching by performing data caching and data delivery to clients in parallel. (ii) Investigate a proactive approach to selecting a repair strategy based on OSD resource availability and the SLA. For example, while RORP recovers more chunks than ROR, it incurs higher encoding costs. (iii) Extend ROR to other EC families. (iv) Extend CephFS to support the repair of single blocks of a PG. This brings several benefits, including flexible per-block lazy repair scheduling, repairing a block soon after it is read, and running partial PG repairs, thereby reducing contention during this operation.

Acknowledgment

This work is supported by the "FrugalCloud" Inria and OVHcloud partnership. Experiments presented in this paper were carried out using the Grid'5000 / Slices testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER, and several Universities as well as other organizations (see http://www.grid5000.fr/).

References

[1] Rodrigo Rodrigues and Barbara Liskov. "High Availability in DHTs: Erasure Coding vs. Replication". In: *Peer-to-Peer Systems IV*. Ed. by Miguel Castro and Robbert van Renesse. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 226–239. ISBN: 978-3-540-31906-1.

[2] John Kubiatowicz, David Bindel, Yan Chen, Steven Czerwinski, Patrick Eaton, Dennis Geels, Ramakrishna Gummadi, Sean Rhea, Hakim Weatherspoon, Westley Weimer, Chris Wells, and Ben Zhao. "OceanStore: an architecture for global-scale persistent storage". In: *SIGARCH Comput. Archit. News* 28.5 (Nov. 2000), pp. 190–201. ISSN: 0163-5964. DOI: 10.1145/378995.379239. URL: https://doi.org/10.1145/378995.379239.

[3] Mohammad Rizk, Shadi Ibrahim, and Thomas Lambert. "ALTOCUMULUS: Enabling Efficient Erasure Coding in IPFS". In: *CCGrid 2026 : The 26th IEEE International Symposium on Cluster, Cloud, and Internet Computing*. Sydney, Australia, May 2026. URL: https://hal.science/hal-05577974.

[4] Jad Darrous and Shadi Ibrahim. "Understanding the performance of erasure codes in hadoop distributed file system". In: *Proceedings of the Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*. CHEOPS '22. Rennes, France: Association for Computing Machinery, 2022, pp. 24–32. ISBN: 9781450392099. DOI: 10.1145/3503646.3524296. URL: https://doi.org/10.1145/3503646.3524296.

[5] Shadi Ibrahim and Jad Darrous. "Erasure Coding Aware Block Placement for Data-Intensive Applications". In: *SIGOPS Oper. Syst. Rev.* 59.1 (Aug. 2025), pp. 62–69. ISSN: 0163-5980. DOI: 10.1145/3759441.3759451. URL: https://doi.org/10.1145/3759441.3759451.

[6] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. "Ceph: a scalable, high-performance distributed file system". In: *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*. OSDI '06. Seattle, Washington: USENIX Association, 2006, pp. 307–320. ISBN: 1931971471.

[7] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. "Erasure Coding in Windows Azure Storage". In: *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. Boston, MA: USENIX Association, June 2012, pp. 15–26. ISBN: 978-931971-93-5. URL: https://www.usenix.org/conference/atc12/technical-sessions/presentation/huang.

[8] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, and Sanjeev Kumar. "f4: Facebook's Warm BLOB Storage System". In: *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. Broomfield, CO: USENIX Association, Oct. 2014, pp. 383–398. ISBN: 978-1-931971-16-4. URL: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/muralidhar.

[9] Maheswaran Sathiamoorthy, Megasthenis Asteris, Dimitris Papailiopoulos, Alexandros G. Dimakis, Ramkumar Vadali, Scott Chen, and Dhruba Borthakur. "XORing elephants: novel erasure codes for big data". In: *Proc. VLDB Endow.* 6.5 (Mar. 2013), pp. 325–336. ISSN: 2150-8097. DOI: 10.14778/2535573.2488339. URL: https://doi.org/10.14778/2535573.2488339.

[10] Hai Zhou, Dan Feng, and Yuchong Hu. "Bandwidth-Aware Scheduling Repair Techniques in Erasure-Coded Clusters: Design and Analysis". In: *IEEE Transactions on Parallel and Distributed Systems* 33.12 (2022), pp. 3333–3348. DOI: 10.1109/TPDS.2022.3153061.

[11] Qiaori Yao, Yuchong Hu, Xinyuan Tu, Patrick P. C. Lee, Dan Feng, Xia Zhu, Xiaoyang Zhang, Zhen Yao, and Wenjia Wei. "PivotRepair: Fast Pipelined Repair for Erasure-Coded Hot Storage". In: *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. 2022, pp. 614–624. DOI: 10.1109/ICDCS54860.2022.00065.

[12] Xiaolu Li, Zuoru Yang, Jinhong Li, Runhui Li, Patrick P. C. Lee, Qun Huang, and Yuchong Hu. "Repair Pipelining for Erasure-coded Storage: Algorithms and Evaluation". In: *ACM Trans. Storage* 17.2 (May 2021). ISSN: 1553-3077. DOI: 10.1145/3436890. URL: https://doi.org/10.1145/3436890.

[13] Tong Liu, Shakeel Alibhai, and Xubin He. "A Rack-Aware Pipeline Repair Scheme for Erasure-Coded Distributed Storage Systems". In: *Proceedings of the 49th International Conference on Parallel Processing*. ICPP '20. Edmonton, AB, Canada: Association for Computing Machinery, 2020. ISBN: 9781450388160. DOI: 10.1145/3404397.3404444. URL: https://doi.org/10.1145/3404397.3404444.

[14] K. V. Rashmi, Nihar B. Shah, and P. Vijay Kumar. "Optimal Exact-Regenerating Codes for Distributed Storage at the MSR and MBR Points via a Product-Matrix Construction". In: *IEEE Transactions on Information Theory* 57.8 (2011), pp. 5227–5239. DOI: 10.1109/TIT.2011.2159049.

[15] Myna Vajha, Vinayak Ramkumar, Bhagyashree Puranik, Ganesh Kini, Elita Lobo, Birenjith Sasidharan, P. Vijay Kumar, Alexandar Barg, Min Ye, Srinivasan Narayanamurthy, Syed Hussain, and Siddhartha Nandi. "Clay Codes: Moulding MDS Codes to Yield an MSR Code". In: *16th USENIX Conference on File and Storage Technologies (FAST 18)*. Oakland, CA: USENIX Association, Feb. 2018, pp. 139–154. ISBN: 978-1-931971-42-3. URL: https://www.usenix.org/conference/fast18/presentation/vajha.

[16] Dimitris S. Papailiopoulos and Alexandros G. Dimakis. "Locally Repairable Codes". In: *IEEE Transactions on Information Theory* 60.10 (2014), pp. 5843–5855. DOI: 10.1109/TIT.2014.2325570.

[17] Dimitri Pertin, Didier Féron, Alexandre Van Kempen, and Benoît Parrein. *Performance evaluation of the Mojette erasure code for fault-tolerant distributed hot data storage*. 2015. arXiv: 1504.07038 [cs.IT]. URL: https://arxiv.org/abs/1504.07038.

[18] Lu Zhong, Guojun Han, Hanxu Hou, and Qinda Hai. "Optimizing Repair-Cost of Locally Repairable Codes for Hot Data in Cluster Storage Systems". In: *2022 IEEE International Conference on Big Data (Big Data)*. 2022, pp. 3235–3241. DOI: 10.1109/BigData55660.2022.10020516.

[19] Mark Silberstein, Lakshmi Ganesh, Yang Wang, Lorenzo Alvisi, and Mike Dahlin. "Lazy Means Smart: Reducing Repair Bandwidth Costs in Erasure-coded Distributed Storage". In: *Proceedings of International Conference on Systems and Storage*. SYSTOR 2014. Haifa, Israel: Association for Computing Machinery, 2014, pp. 1–7. ISBN: 9781450329200. DOI: 10.1145/2611354.2611370. URL: https://doi.org/10.1145/2611354.2611370.

[20] Ranjita Bhagwan, Kiran Tati, Yu-Chung Cheng, Stefan Savage, and Geoffrey M. Voelker. "Total recall: system support for automated availability management". In: *Proceedings of the 1st Conference on Symposium on Networked Systems Design and Implementation - Volume 1*. NSDI'04. San Francisco, California: USENIX Association, 2004, p. 25.

[21] Frederic Giroire, Julian Monteiro, and Stéphane Pérennes. "Peer-to-Peer Storage Systems: A Practical Guideline to be Lazy". In: *2010 IEEE Global Telecommunications Conference GLOBECOM 2010*. 2010, pp. 1–6. DOI: 10.1109/GLOCOM.2010.5683761.

[22] Chuang Gan, Yuchong Hu, Leyan Zhao, Xin Zhao, Pengyu Gong, and Dan Feng. "Revisiting network coding for warm blob storage". In: *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*. FAST '25. USA: USENIX Association, 2025. ISBN: 978-1-939133-45-8.

[23] Jad Darrous, Shadi Ibrahim, and Christian Perez. "Is it Time to Revisit Erasure Coding in Data-Intensive Clusters?" In: *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. 2019, pp. 165–178. DOI: 10.1109/MASCOTS.2019.00026.

[24] Daniel Balouek, Alexandra Carpen Amarie, Ghislain Charrier, Frédéric Desprez, Emmanuel Jeannot, Emmanuel Jeanvoine, Adrien Lèbre, David Margery, Nicolas Niclausse, Lucas Nussbaum, Olivier Richard, Christian Pérez, Flavien Quesnel, Cyril Rohr, and Luc Sarzyniec. "Adding Virtualization Capabilities to the Grid'5000 Testbed". In: *Cloud Computing and Services Science*. Ed. by Ivan I. Ivanov, Marten van Sinderen, Frank Leymann, and Tony Shan. Vol. 367. Communications in Computer and Information Science. Springer International Publishing, 2013, pp. 3–20. ISBN: 978-3-319-04518-4. DOI: 10.1007/978-3-319-04519-1_1.

[25] *CephFS LRC Documentation — Ceph Documentation*. https://docs.ceph.com/en/latest/rados/operations/erasure-code-lrc/. 2026.

[26] *Cloud Disk Array — ovhcloud.com*. https://www.ovhcloud.com/fr/storage-solutions/cloud-disk-array/. 2026.

[27] *IBM Storage Ceph — ibm.com*. https://www.ibm.com/products/storage-ceph. 2026.

[28] Sage Weil, Scott Brandt, Ethan Miller, and Carlos Maltzahn. "CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data". In: *ACM/IEEE SC 2006 Conference (SC'06)*. Tampa, FL, USA: IEEE, 2006, pp. 31–31. ISBN: 978-0-7695-2700-0. URL: http://ieeexplore.ieee.org/document/4090205/.

[29] Runzhou Han, Chao Shi, Tabassum Mahmud, Zeren Yang, Vladislav Esaulov, Lipeng Wan, Yong Chen, Jim Wayda, Matthew Wolf, and Mai Zheng. "Revisiting Erasure Codes: A Configuration Perspective". In: *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*. Santa Clara CA USA: ACM, 2024, pp. 93–100. ISBN: 9798400706301. URL: https://dl.acm.org/doi/10.1145/3655038.3665951.

[30] Ilias Iliadis. "Reliability Evaluation of Erasure-coded Storage Systems with Latent Errors". In: *ACM Trans. Storage* 19.1 (Jan. 2023). ISSN: 1553-3077. DOI: 10.1145/3568313. URL: https://doi.org/10.1145/3568313.

[31] *Intel Intelligent Storage Acceleration Library Homepage*. 2026. URL: https://software.intel.com/en-us/storage/ISA-L.

[32] Bin Fan, Wittawat Tantisiriroj, Lin Xiao, and Garth Gibson. "Diskreduce: Replication as a prelude to erasure coding in data-intensive scalable computing". In: *Carnegie Mellon University Parallel Data Laboratory, Tech. Rep. (cmu-pdl-11-112)* (2011).

[33] Mingyuan Xia, Mohit Saxena, Mario Blaum, and David A. Pease. "A tale of two erasure codes in HDFS". In: *Proceedings of the 13th USENIX Conference on File and Storage Technologies*. FAST'15. Santa Clara, CA: USENIX Association, 2015, pp. 213–226. ISBN: 9781931971201.

[34] Loic Guegan and Anne-Cécile Orgerie. "Estimating the End-to-End Energy Consumption of Low-Bandwidth IoT Applications for WiFi Devices". In: *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2019, pp. 287–294. DOI: 10.1109/CloudCom.2019.00049.

[35] K. V. Rashmi, Mosharaf Chowdhury, Jack Kosaian, Ion Stoica, and Kannan Ramchandran. "EC-cache: load-balanced, low-latency cluster caching with online erasure coding". In: *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*. OSDI'16. Savannah, GA, USA: USENIX Association, 2016, pp. 401–417. ISBN: 9781931971331.

[36] Kevin Hsieh, Aaron Harlap, Nandita Vijaykumar, Dimitris Konomis, Gregory R. Ganger, Phillip B. Gibbons, and Onur Mutlu. "Gaia: geo-distributed machine learning approaching LAN speeds". In: *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation*. NSDI'17. Boston, MA, USA: USENIX Association, 2017, pp. 629–647. ISBN: 9781931971379.

[37] Theophilus Benson, Aditya Akella, and David A. Maltz. "Network traffic characteristics of data centers in the wild". In: *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement*. IMC '10. Melbourne, Australia: Association for Computing Machinery, 2010, pp. 267–280. ISBN: 9781450304832. DOI: 10.1145/1879141.1879175. URL: https://doi.org/10.1145/1879141.1879175.

[38] Chuang Gan, Yuchong Hu, Leyan Zhao, Xin Zhao, Pengyu Gong, and Dan Feng. "Revisiting network coding for warm blob storage". In: *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*. FAST '25. Santa Clara, CA, USA: USENIX Association, 2025. ISBN: 978-1-939133-45-8.